Big Chalk Mix Engine

Automated Regression Engine & Dashboard — Complete Feature Reference

Every capability currently built into the system: what it does, how to use it, the method behind it, and why it was designed that way. Written to serve both as an operating manual and as the methodology record for the capstone.

Project	Northwestern MSDS Capstone, Summer 2026
Sponsor	Big Chalk Analytics — Alex Hathcock, Arko
Faculty	Abid Ali
Team	Feifan Liu, Boqi (Jerry) Niu, Jiahao (Edison) Li
Objective	Automated regression models predicting Volume Sales for every Brand × Retailer-Channel combination, with a dashboard for inspection and manual tuning
Dataset	156 weeks (Jan 2023 – Dec 2025), 10 brands, 9 channels, 113 columns
Status	76 models built, 6 slices deliberately not modeled
Document date	August 10, 2026

How to read this document

Sections 1–2 describe the modeling engine and its methods. Section 3 walks the dashboard screen by screen. Sections 4–6 are reference tables for configuration fields, export files and command-line tools. Sections 7–9 report current results, the reasoning behind the significant design decisions, and what is verified. Section 10 states the known limits honestly.

Contents

1 System overview

1.1 What the system does

The engine fits one regression model per **Brand × Channel** slice, explaining weekly Volume Sales from marketing and market drivers: distribution, price, trade promotion, media, competitive activity, seasonality and macroeconomics. It runs the full set of slices unattended, grades each result, and decomposes every model into weekly driver contributions that reconcile exactly to the fitted line.

Around the engine sits a six-screen dashboard for inspecting, tuning and exporting those models. The design goal stated by the sponsor is a system an analyst can point at a data file and get a defensible model set from, without hand-building each model — while still being able to override any decision the automation made.

1.2 Architecture

Component	File	Responsibility
Modeling engine	code/capstone_pipeline.py	All statistics: loading, windowing, transforms, variable selection, constrained fitting, contributions, due-tos, curve optimization. No UI code. Importable and testable on its own.
Dashboard	code/dashboard.py	Plotly Dash application: six screens, the results cache, Excel export, config editing and template round-trip.
Batch runner	code/run_all.py	Command-line run over every slice; writes per-slice CSVs plus summary, skipped and failure files.
Single-slice runner	code/run_brand1_channel1.py	Focused run of one slice for debugging.
Equivalence check	code/reference_alex_curvefit.py	Proves our constrained solver matches the sponsor's own curve_fit approach numerically.
Scaling test	code/scaling_test.py	Runtime scaling measurements.
Variable configs	configs/*.csv	Per-product and per-product-per-channel modeling configuration. The CSV, not the code, is the source of truth.

1.3 Running it

```
# dashboard
cd code && python dashboard.py        # http://127.0.0.1:8050

# full batch from the command line
cd code && python run_all.py
cd code && python run_all.py 1 3      # brands 1-3 only (resumable chunks)
```

Requirements are pinned in `requirements.txt`. The dashboard accepts any data file path in the left rail; the command-line scripts expect `Anonymized Data for Project.xlsx` in the repository root.

2 The modeling engine

2.1 Slice definition and data loading

A **slice** is one Brand (an Excel sheet) × one Channel (the Geography column). Weeks are parsed from strings of the form `WE 01/08/2023` into real dates and sorted. Every model is fit on exactly one slice; there is no pooling across brands or channels.

Columns that are algebraic decompositions of the target — anything beginning `Volume Sales` or `Dollar Sales`, such as 'Volume Sales Any Promo' — are excluded as predictors automatically. They are components of the thing being predicted, so including them would produce a model with a perfect fit and no meaning. The exclusion follows the target: change the target and the decompositions of the *new* target are the ones removed.

2.2 The fixed calendar window

This fixed a real bug found by the sponsor

The window used to be 'the last N **rows** this slice happens to have'. A product delisted in mid-2023 therefore got its own private 2023 window, still produced a model, and leaked 18-month-old due-tos into the export as a disjoint chunk of history. Alex found it by pivoting the exported weekly due-tos in Excel.

> *"The window should be predefined, so the last 104 weeks should be the same 104 weeks for everything. Instead of having the window being dependent on the data, the data is dependent on the window." — Alex, 2026-07-27*

The window is now an **absolute date range**. It ends at the latest week present anywhere in the data file (the *anchor*) and runs back 52, 104 or 156 weeks. It is identical for every slice, so models are comparable and no slice can be modeled on stale history.

Window	Calendar range on the current dataset
52 weeks	`Jan 05, 2025 – Dec 28, 2025`
104 weeks (default)	`Jan 07, 2024 – Dec 28, 2025`
156 weeks	`Jan 08, 2023 – Dec 28, 2025`

The anchor is computed once per data file and cached by file path, modification time and size, so repeated runs do not re-read the workbook.

Slices that are not modeled

A slice needs a minimum amount of data *inside* the window to be worth modeling. Below either threshold it is not modeled at all — it raises a typed exception carrying the counts, and is reported as skipped with the reason. It never appears as a model, and it never contributes a due-to.

Threshold	Default	Meaning
`min_weeks`	52	Weeks of any data inside the window

Threshold	Default	Meaning
`min_nonzero_weeks`	26	Weeks with non-zero target (actual selling weeks)

Skipped slices surface in three places: a **Not modeled** table on the Model Runs screen, the `errors` sheet of the Excel export, and `all_models_skipped.csv` from the command-line runner. Nothing disappears silently — the point is that a data problem should be visible, not invisible.

2.3 Two-tier variable configuration

Predictors are not hard-coded. A CSV per product maps each raw column to a family, an expected sign, optional bounds, transforms and a role. An optional second file overrides it for a single channel.

Tier	File	Applies to
Product default	`configs/varconfig_<dataset>_<brand>.csv`	Every channel of that brand
Product × Channel override	`configs/varconfig_<dataset>_<brand>__ch_<channel>.csv`	That one channel — **overrides wins**

One resolver function is shared by the dashboard, the batch runner and the single-slice runner, so all three agree on which config a given slice uses. Every config write goes through a single process lock and a transactional temp-file-plus-rollback commit, so a failed edit cannot leave a half-written config on disk.

ACV Weighted Distribution is force-included by default. This lives in the generated config's `role` column rather than in code, so it is visible and the analyst can override it back to `auto`. Justification: a volume model with no distribution term cannot track shelf-presence change; on Brand 1 × Channel 1 forcing it moved holdout MAPE from 73% to 32%.

2.4 Transforms

Applied in this order, matching the convention used by Meta's Robyn and Google's Meridian:

```
raw execution  →  adstock (carry-over)  →  scale (units)  →  Hill (saturation)
```

Adstock — carry-over

$$a(t) = (1 - d) \cdot x(t) + d \cdot a(t-1)$$

The (1−d) share of this week's execution lands this week; the remainder carries forward. The formulation is **normalized**: the adstocked total approximately equals the raw total. The naive $a(t) = x(t) + d \cdot a(t-1)$ inflates total impressions, which in turn inflates the media contribution — a quiet way to overstate media's value.

> *"If you see an ad and you wait to do grocery shopping until next week, it's going to have a decayed amount." — Alex*

Decay is per variable. Suggested starting values: search-type media ≈ 0.2 (acted on immediately), TV/CTV ≈ 0.7 (long memory), 0.5 default. A diagnostic reports the adstocked-versus-raw total for each media variable so the normalization can be checked.

Scale — readable units

Divides a variable before modeling so its coefficient reads 'per 1,000 impressions' rather than 'per impression'. This is a **units change only**: the coefficient shrinks by exactly the factor the data does, so every contribution, due-to and fitted value is unchanged (verified to 1.2×10^{-10}).

Custom coefficient bounds are written in **original** units and transformed with the scale internally. Without that, setting a scale of 1,000 would silently tighten every custom bound by a factor of 1,000. Sign constraints need no such treatment, since dividing by a positive number cannot flip a sign.

Hill — saturation

Identifying the curve

The sponsor described this curve from memory: 'some Bill and this other data scientist created this for the efficacy of drugs … a curve that has a midpoint and a slope, so it makes it sigmoidal'. That is the **Hill equation** — A. V. Hill, 1910, fitting oxygen binding to haemoglobin, and since then the standard dose–response curve in pharmacology. It is also the saturation function in both Robyn and Meridian. Two parameters, a midpoint and a slope, exactly as described.

$$H(x) = x^s / (x^s + k^s), \qquad k = \gamma \cdot ref$$

Parameter	Config column	Meaning
γ — midpoint	`sat_midpoint`	Half-saturation point as a *fraction* of the reference scale. At x = γ·ref the response is exactly half its maximum. Stored as a fraction so the parameter transfers across brands and channels whose spend differs by orders of magnitude. Valid range (0, 1].
s — slope	`sat_slope`	Shape. s > 1 gives the S-curve: slow start, steep through the midpoint, then flattening. s < 1 diminishes straight from the origin, which weekly aggregate data often prefers.

Parameter	Config column	Meaning
ref	`(computed)`	The scale the midpoint is a fraction of — the peak adstocked, scaled execution. **Computed on training weeks only**, otherwise the transform itself would see the holdout.

Because H returns a value in [0, 1), the coefficient on a saturated variable is the **maximum weekly volume that variable can deliver** — a directly useful number. Contributions remain additive, so the due-to decomposition is unaffected (verified exact to 1.0×10^{-10}).

Saturation is **off by default**: both parameters blank means the variable stays linear. Half a specification is treated as no saturation rather than guessed at.

2.5 Automated variable selection

Three stages, run on standardized data so the selection is scale-invariant:

Stage 1 — VIF pruning

Iteratively drops the predictor with the highest variance inflation factor until all are below the threshold. VIFs are computed at once from the diagonal of the inverse correlation matrix — mathematically identical to running the k auxiliary regressions, roughly 100× faster on a wide matrix. Force-included variables are protected and never dropped.

Why VIF rather than pairwise correlation or PCA: VIF catches a variable that is redundant with a *combination* of others, which a correlation matrix misses; and unlike PCA it keeps the raw, explainable variables the client needs to see.

Stage 2 — forward stepwise with family caps

Adds the variable with the lowest p-value while additions remain significant at `p_enter`. Forced variables are seeded in first.

Why stepwise rather than Lasso or tree importance: every addition is a single testable decision that can be explained to a client. A Lasso path is harder to defend line by line, and tree importances do not give the signed, bounded coefficients the deliverable requires.

Per-family caps limit how many variables of each family may enter. The cap is enforced *during* selection, not by trimming afterwards, so a capped family spends its budget on its most significant members while the other families still fill normally. Forced variables are always kept and count toward the budget: a cap of 2 with 2 forced Trade variables admits no further Trade.

> *“Is there a way to have variable hyperparameter tuning? We want to be more restrictive, we want to be less restrictive … more restrictive by a group. I want only two to three trade, but I want all media.” — Alex*

Stage 3 — dead-coefficient pruning

After the constrained fit, any variable whose standardized impact (|coefficient| × standard deviation) is under 1×10^{-4} of the largest is removed and the model refit. These are variables the bounded solver has pushed to effectively zero; leaving them in would inflate the predictor count and clutter the coefficient table without changing the fit.

Strictness presets

Exposed on the Variables screen as a single control. Balanced reproduces the engine's historical defaults exactly, so results are unchanged until someone opts in.

Preset	p_enter	VIF threshold	Avg predictors per model (76 slices)
Punitive	0.01	5	4.4
Balanced (default)	0.05	10	6.5
Lax	0.15	20	10.0

2.6 Constrained final fit

The final model is bounded least squares (`scipy.optimize.lsq_linear`, trust-region reflective) in original units, so coefficients are interpretable and contributions decompose additively.

Guardrail	Behaviour
Expected sign	A positive-signed variable can never enter with a negative coefficient, and vice versa. The sign is a hard constraint, not a preference.
Custom bounds	Explicit [lo, hi] limits per coefficient; these override the sign default. lo must be strictly < hi.
Intercept	Always unconstrained, and reported as its own due-to line.

Why bounded least squares rather than Ridge: Ridge cannot honour sign or box constraints, and the sponsor requires them — a model that says price increases raise volume is not deliverable regardless of its fit statistics. Bounded least squares enforces any [lo, hi] per coefficient while remaining a transparent linear model.

An unconstrained OLS is fit alongside purely to supply t-statistics as an inference reference, and VIFs are recomputed on the final in-model set.

Sign-conflict diagnostic

When the expected sign is positive but the unconstrained t-statistic is below −1.96 (or the mirror case), the variable is flagged as a sign conflict. The constraint still holds — but the analyst is told that the data disagrees with the prior, which is usually a signal about the data rather than about the prior.

2.7 Validation design

This is the part of the engine that determines whether any reported accuracy number can be trusted, so it is documented in full.

One structure, two fits

Within the fixed window, the last up to 13 weeks are **always** reserved as a validation tail (floored to keep at least 5 training weeks; set to 0 to disable).

- **Validation model** — selects the variable structure on the window *before* the tail, so selection never sees the tail, and is scored on the tail. That score is the reported **holdout MAPE**.
- **Reported model** — refits that *same* structure on the full window including the tail. These are the coefficients and due-tos displayed.

So the holdout validates the same structure and the same modeling process out of sample, and the reported metric is a genuine out-of-sample number — never the all-data in-sample fit relabelled. Both fits live inside the same fixed calendar window: the tail is carved out of it rather than the window sliding backwards, so the validation model never reaches for data outside the period being modeled.

Why a time-based holdout rather than random K-fold: weekly series are autocorrelated, and random folds leak the future into training. A model validated by random folds on time-series data will look far better than it is.

Three-way split for curve optimization

When the media curve optimizer runs, a two-way split is not enough. Searching a large space over-fits whatever it is scored on — that is arithmetic, not bad luck. So the window is split three ways:

```
[ ——— train ——— | inner CV folds | OUTER HOLDOUT ]
                                                13 wks
```

Candidates are scored by **rolling-origin cross-validation** on the inner folds — train on everything before a fold, score the fold, average across three expanding-window folds. The outer holdout is never touched during the search, so the holdout MAPE reported afterwards remains an honest number, directly comparable with a model that was never optimized.

> **Why this matters more than it sounds**
>
> Had the optimizer been scored on the reported holdout, every optimized model would have looked better and not one of them would have been. The three-way split is what makes it possible to state honestly, in section 7, that the optimizer's gains often do not transfer.

2.8 Fit metrics and grading

Metric	Definition
R^2 / adjusted R^2	Share of weekly variance explained; adjusted R^2 penalizes added predictors. Shown together so a high R^2 from over-fitting is visible.

Metric	Definition
In-sample MAPE	Mean absolute percentage error on the training window, over non-zero-actual weeks only (zero-sales weeks would divide by zero in partial-distribution channels).
Holdout MAPE	Out-of-sample error on the reserved validation tail. This is the number that grades the model.
Durbin–Watson	Residual autocorrelation check; approximately 2 means no autocorrelation.
t-statistic	From the unconstrained OLS as an inference reference; $\|t\| \geq 2$ is roughly significant at 5%.
VIF	Collinearity on the final in-model variables. Above 10 is flagged as a non-blocking warning; forced variables can be high by design.

Grade	Holdout MAPE	Colour
Good Model	≤ 10%	Green
Moderate Model	> 10% and ≤ 40%	Amber
Bad Model	> 40%	Red
No holdout	No holdout period available to score	Grey

These cutoffs are a team convention reusing the review-flag threshold and should be confirmed with the client.

2.9 Contributions and due-tos

> **These are two different numbers**
>
> The tool previously showed only the first under the second's name. Arko raised it and Alex confirmed: ‘what you have here is contribution — what is the total per period. The due-to is just the change … that's really what all the people that look at this actually care about: how is the change of macroeconomic variables impacting my sales year to year.’

Term	Type	Definition
Contribution	Level	How much volume a driver accounts for *in* a period: the sum of coefficient × value over that period's weeks.
Due-to	Change	How much of the movement *between* two periods a driver explains: its contribution in period B minus its contribution in period A.

The decomposition is exact by construction. Intercept plus the sum over drivers of coefficient × value reproduces the fitted value every week, with the intercept reported as its own line. Verified to 5.8×10^{-11}; due-tos sum to the change in modeled volume to 6×10^{-9}.

Unequal-length periods

Periods of different lengths are compared as **per-week averages** automatically. Comparing totals would make a 52-week period beat a 4-week one for reasons that have nothing to do with the drivers. The caption states which basis is in use.

Averaging rule for media

Average weekly contribution for media — any variable with an adstock decay — is taken **only over weeks with real execution** (non-zero raw spend), never diluted by the zero weeks of a flight that started mid-window. Non-media drivers average over all weeks. This rule was requested explicitly by the sponsor.

Model years

Consecutive 52-week blocks are labelled counting back from the latest week, so the most recent block is the current model year. Labels carry their date range, for example ‘Year 2 (Jan 2025-Dec 2025)’. Current and Previous always map to the latest and prior model year regardless of window length.

2.10 Comparable coefficients

> *“You can't really compare the coefficient for ACV weighted distribution versus the five-year inflation expectations or gas price, because the support that you're using to build the model is not comparable.” — Arko, 2026-07-27*

Raw coefficients cannot be ranked across variables, because the inputs sit on different supports: a savings rate moves between 2 and 8, impressions between zero and millions. The engine therefore reports two derived quantities that put every driver in the **same unit — volume**:

Quantity	Formula	Reads as
Impact / SD	β × SD(x)	Volume moved by a one-standard-deviation change in this variable
Impact / range	β × (max − min)	Volume moved across the variable's full observed range

Both are invariant to the scale setting, so the comparison is stable however the analyst sets units. Worked example, Brand 1 × Channel 1:

Variable	Raw coefficient	Impact / SD
Seasonality Index	61,473	13,193
ACV Weighted Distribution	3,107	13,151

The raw coefficients look twenty times apart. In volume terms the two drivers are effectively tied. **Impact / SD is the column that can legitimately be ranked.**

Why not min-max normalization

Arko asked whether the data could be normalized before modeling. For an unregularized constrained least-squares fit, min-max normalization is a linear re-parameterization — as he himself noted, ‘it does nothing to your model’. Selection already standardizes internally, so it was scale-invariant already. Normalizing and then inverting every reported number would introduce exactly the risk he warned about — ‘when you report the results you have to go back to the original scale’ — while changing no result. So the actual problem, comparability, is solved directly instead.

2.11 Media curve optimization

“Does it always have to be 0.5? Can we create an optimization engine that will pick the best ad stock based on correlation with the data, predictive power in a regression? We can do the same thing with saturation. If we want a fully automated system, that would be dope as hell.” — Alex

Searches adstock decay and both Hill parameters for each media variable in a slice's model.

Aspect	Design
Search space	decay ∈ {0, 0.2, 0.4, 0.6, 0.8} × midpoint ∈ {0.2, 0.35, 0.5, 0.65, 0.8} × slope ∈ {0.5, 1.0, 1.5, 2.5}, plus the option of no saturation — so the search can always decline to saturate.
Strategy	Coordinate descent: one variable at a time, others held fixed, up to two passes, stopping early on convergence. A full joint search is grid^k in the number of media variables; coordinate descent is k×grid per pass and finds the same optimum on a near-separable problem.
Scoring	Mean MAPE across three rolling-origin cross-validation folds, all of which end before the outer holdout.
Adoption rule	A candidate must not be worse in **any** fold (strict dominance) *and* must improve the mean by at least 0.5 percentage points. Without the floor the search would trade a 0.01% gain for an implausible curve shape.
Structure	Held fixed during the search, so the score reflects curve shapes only — and so the search stays fast.
Output	Parameters only. The optimizer writes candidates into the config table; it does not save and does not run. Nothing here becomes a second, divergent modeling path.

Runtime is approximately 1.7 seconds per slice on average, up to about 9 seconds for a slice with three media variables — roughly 4 minutes for all 76 slices.

Section 7.3 reports what the optimizer actually achieves, including where it does not help.

3 The dashboard, screen by screen

3.1 Global elements

Left rail

- **Workspace navigation** — six screens: Diagnostics, Variables, Contributions, Model Runs, Export, Definitions.
- **Dataset panel** — type any data file path and press *Load data*. Shows the loaded file and a status line. Product and channel lists populate from the file.

Top bar — results filter only

The top bar carries **Product** and **Channel** and nothing else that can be edited. These are filters over already-computed results: changing them re-reads the cache and does not re-run anything.

Target and window are *modeling* decisions, so they live in exactly one place — the Variables screen — and are echoed here read-only with the actual calendar range, for example ‘Volume Sales · 104 wks · Jan 07, 2024 – Dec 28, 2025’.

> *“Remove everywhere that you can change the target. You should only be able to change the target in one spot … The only thing you should be filtering on are results, which would be your product and channel.” — Alex*

A run-status indicator on the right reports whether the current view came from a fresh run or from the cache.

The results cache — run once, filter many

> *“Instead of having to rerun every time, you just create the data that filters into this for everything, and then you just filter down. It's going to take a little bit longer for the first one, but then when we're evaluating, it's instantaneous.” — Alex*

A batch run computes every slice once and caches the full results. Diagnostics, Contributions and Export then read and filter that cache. Measured: a full batch is about 19 seconds for 76 models; a filter change is about 140 milliseconds.

The cache is keyed by **data file, target, window, strictness and family caps**. Change any modeling knob and the screens ask for a fresh batch rather than showing a blend of two settings. This also removes a hazard the sponsor named directly — ending up with one model on 156 weeks while everything else on screen is on 104. Re-tuning a single slice on Variables and pressing Run refreshes just that entry.

If a slice is not in the cache, the screen says why — not yet run, skipped for insufficient data with the reason, or failed — rather than showing the previous slice's numbers, which would be actively misleading.

3.2 Diagnostics

Everything needed to judge a single model.

Element	What it shows
Metric strip	R^2, adjusted R^2, in-sample MAPE, holdout MAPE, Durbin–Watson, predictor count with the forced count. Holdout MAPE above 40% carries a REVIEW flag.
Actual vs fitted	Weekly actual and fitted lines over the window, with the reserved holdout period shaded and the out-of-sample forecast drawn separately. The caption states the slice, target and exact window dates.
Residuals vs fitted	Scatter for heteroscedasticity and structure.
Residual distribution	Histogram for skew and outliers.
Coefficient table	One row per in-model variable plus the intercept: family chip, expected sign, t-statistic with a visual bar, VIF, coefficient (with its scale divisor when set), **Impact / SD**, and average weekly contribution. The caption reports how many of how many candidates were selected, sign-conflict count and high-VIF count.
Media response curves	Fitted response curve per media variable with the weeks actually executed plotted on it, and the half-saturation point marked.

Reading the response curve chart

- **A flattening curve** is a diminishing return. The dotted vertical is the half-saturation point: past it, the next unit of execution returns less than the last.
- **A straight line** means no saturation was fitted; the variable is linear in execution.
- **Where the dots sit** is the current operating point. Clustered on the steep part means headroom; out on the flat means extra spend is buying little.

3.3 Variables

The only screen where modeling decisions are made.

Modeling settings

Target (any raw column may be the dependent variable; Volume Sales, Dollar Sales and Unit Sales are offered first) and window (52 / 104 / 156 weeks), with the resolved calendar range displayed alongside.

Variable selection controls

Strictness preset, and an editable **max per family** table covering Distribution, Trade, Media, Competitive, Macro, Price, Category Price, Seasonality and Trend. Blank means no limit; blank, zero and unparseable entries are all treated as no limit so a half-filled table cannot accidentally throttle a family. A live note reports the effective settings.

Configuration scope

Edit the Product default (all channels) or a Channel-specific override. A status line states which file is being edited and whether an override exists. **A Variables-tab Run is WYSIWYG**: it runs exactly

the config on screen, which can differ from what the batch would resolve.

The variable table

One row per candidate variable, with all of the columns documented in section 4, plus three read-only context columns showing the current coefficient, Impact / SD and contribution for the last run — gated to the exact Product × Channel × Target of that run, so switching slices blanks them rather than showing another slice's numbers. Rows with role *force* are tinted blue and *exclude* rows pink.

Actions

Control	Effect
Due-to period	Which period the context column shows (Y1 / Y2 / Total).
Hide excluded	View filter only. The underlying data stays complete, so excluded rows are never dropped on save.
Upload template	Apply a filled template (.xlsx/.csv). Rows with channel=ALL write the Product default; a specific channel writes that override. A variable in the data but not listed for a group becomes excluded.
Download template	This product's config (default plus every existing override) as an editable file.
Reset to default	In a channel scope, delete that channel's override so it inherits the Product default again.
Regenerate defaults	Rebuild the config from the data's naming heuristics.
Save config	Validate and write. Bounds with lo ≥ hi are rejected with the offending variables named.
Optimize media curves	Run the curve search and write candidates into the table. Reports the cross-validated gain. Does not save and does not run.
Run model	Persist the on-screen edits, fit this slice, refresh its cache entry and jump to Diagnostics.

Template uploads are **atomic**: every group is validated first and nothing is written unless all pass, so a later failure cannot leave a partial edit. Unknown variable names are rejected rather than silently excluding everything.

3.4 Contributions

Element	What it shows
Contribution totals by model year	Grouped horizontal bars per driver per model year.
Period panel	A driver chart with a mode toggle — **Contribution** (level in the selected period) or **Due-to** (change between the two selected periods) — plus a primary period and a comparison period.
Contributions by model year table	Signed sums per model year with the year-over-year change and a percentage pill.
Time map upload	A .csv/.xlsx with a date (or week) column and a period column defines custom periods, which then appear in the period pickers.

Built-in periods: entire window, current model year, previous model year, calendar Q4 of the latest year, latest 13 weeks, latest 4 weeks, plus any uploaded mapped periods that overlap the run. The comparison list excludes the currently selected primary period, so a period can never be compared with itself. A period that no longer exists for the current run (for example ‘previous year’ in a 52-week window) falls back to a valid option rather than rendering mislabelled.

In due-to mode the comparison period is the baseline and the primary period is the focus, so the chart reads ‘what changed getting *to* the period I selected’. The caption names both periods, the week counts, whether the basis is totals or per-week averages, and the total modeled change.

3.5 Model Runs

Element	What it shows
Summary pills	Model count, median R^2, median holdout MAPE, count needing review, the window range, the count not modeled, active caps and strictness if not default, and the run duration.
Model table	Every slice: grade (fully coloured green/amber/red), product, channel, target, weeks, selling weeks, predictors, forced count, R^2, in-sample MAPE, holdout MAPE and sign conflicts. Sortable and filterable; click a row to load that slice.
Not modeled table	Every skipped slice with weeks in window, selling weeks and the reason.
Run all combinations	Build the full results cache.
Run model	Re-run the currently selected slice only.
Reload summary	Re-read the saved summary CSV from disk.

3.6 Export

One Excel workbook, either for the current slice or for every combination. Served from the cache when available, so the export cannot silently disagree with what the screens show. Sheets are documented in section 5.

3.7 Definitions

An in-app reference covering fit metrics, grading, contribution versus due-to, variable roles and configuration, media curves, selection hyperparameters, and the window and validation method. It tracks the engine's actual behaviour, so it is the fastest way for a new user to check what a number on screen means without leaving the tool.

4 Configuration reference

4.1 Variable config columns

Column	Values	Meaning
`variable`	column name	The raw data column. Not editable in the UI.
`family`	text	Grouping used for chips, per-family caps and reporting. Standard families are listed in 3.3.
`sign`	positive / negative / unconstrained	Hard guardrail on the coefficient's direction.
`adstock_decay`	0 – 1, or blank	Carry-over decay. Blank means no adstock; any variable with a decay is treated as media for averaging rules.
`scale`	> 0, default 1	Divide the variable before modeling. Units only; never changes the fit.
`sat_midpoint`	(0, 1], or blank	Hill half-saturation as a fraction of peak execution.
`sat_slope`	> 0, or blank	Hill shape. Both saturation fields must be set, or neither.
`coef_lower`	number or blank	Hard lower bound in original units; overrides the sign default.
`coef_upper`	number or blank	Hard upper bound; must be strictly greater than coef_lower.
`role`	auto / force / exclude	**auto**: the model decides. **force**: always in, bypassing selection and VIF pruning. **exclude**: never enters.

4.2 Template columns for upload and download

The template adds `product` and `channel` to the front of the config columns. Channel `ALL` means the Product default; any other value writes that channel's override.

```
product, channel, variable, family, sign, role,
coef_lower, coef_upper, adstock_decay, scale, sat_midpoint, sat_slope
```

4.3 Run-level settings

Setting	Default	Meaning
`target`	`Volume Sales`	The dependent variable.
`model_weeks`	`104`	Window length in weeks.
`window_end`	`dataset anchor`	The shared week every window ends on.
`min_weeks`	`52`	Minimum weeks in window to model a slice.
`min_nonzero_weeks`	`26`	Minimum selling weeks in window.
`holdout_weeks`	`13 (auto)`	Reserved validation tail; 0 disables.

Setting	Default	Meaning
`max_per_family`	`{} (none)`	Per-family entry caps.
`vif_threshold`	`10.0`	VIF pruning threshold.
`p_enter`	`0.05`	Stepwise entry significance.
`default_media_decay`	`0.5`	Fallback adstock decay.
`force_include / exclude`	`[]`	Legacy overrides; roles in the config are the preferred mechanism.

5 Export file reference

A single workbook with four sheets, plus an errors sheet when anything was not modeled. The sponsor's stated workflow is to pull these into a pivot table, so the weekly sheet carries its period columns rather than requiring a lookup.

5.1 fit_statistics

```
product, channel, target, window_weeks, window_start, window_end,
weeks_modeled, selling_weeks, R2, adj_R2, MAPE_in_pct,
MAPE_holdout_pct, grade, n_selected, n_forced
```

The grade cell is filled green, amber or red with white bold text.

5.2 fit_structure

```
product, channel, variable, family, sign, role, coef_lower,
coef_upper, adstock_decay, scale, sat_midpoint, sat_slope,
current_coefficient, in_last_model
```

This is the upload template format plus two read-only context columns requested by the sponsor — `current_coefficient` and `in_last_model`. Both are ignored on re-upload, so an exported model can be tweaked in Excel and re-uploaded without breaking the round trip.

5.3 weekly_due_tos

```
product, channel, date, model_year, period, quarter, year,
driver, due_to
```

Long format, one row per week per driver, including the intercept line so the values reconcile exactly to the fitted series. The `period` column carries labels from an uploaded time map. The date range is exactly the modeling window — no pre-window history can appear here.

5.4 due_to_change

```
product, channel, period_from, period_to, weeks_from, weeks_to,
basis, driver, due_to_change, share_of_total_change_pct
```

Period-over-period due-tos, built for the model-year pair and for consecutive uploaded time-map periods. The `basis` column states whether the comparison used totals or per-week averages.

5.5 errors

Present only when something was not modeled: product, channel and the reason, including insufficient-data-in-window messages with their counts.

6 Command-line tools

Script	Purpose	Outputs
`run_all.py`	Batch every slice with the shared fixed window. Accepts a brand range for resumable chunks.	outputs/all/<slice>_coefficients.csv, <slice>_contrib_by_year.csv, all_models_summary.csv, all_models_skipped.csv, all_models_failures.csv
`run_brand1_channel1.py`	Single-slice run for debugging.	Console
`reference_alex_curvefit.py`	Verifies our bounded least squares matches the sponsor's curve_fit implementation.	PASS/FAIL with the maximum relative difference
`scaling_test.py`	Runtime scaling measurements.	Console / CSV

BLAS thread counts are pinned before NumPy loads in the batch runners: many small least-squares solves thrash badly with many threads, because each solve is tiny and spawn overhead dominates.

7 Current results

7.1 Model set

Measure	Value
Slices modeled	76
Slices not modeled	6
Failures	0
Modeling window	104 weeks, Jan 07 2024 – Dec 28 2025 (identical for every slice)
Median R^2	0.881
Median holdout MAPE	11.5%
Grades	35 Good, 35 Moderate, 6 Bad
Full batch runtime	~19 seconds

7.2 Slices deliberately not modeled

Slice	Weeks in window	Selling weeks	Reason
Brand 1 × Channel 8	0	0	Data ends June 2023, entirely before the window
Brand 2 × Channel 8	0	0	Data ends June 2023, entirely before the window
Brand 1 × Channel 6	104	24	Almost no selling weeks — this was the 254% MAPE model the sponsor flagged
Brand 2 × Channel 5	104	11	Almost no selling weeks

Slice	Weeks in window	Selling weeks	Reason
Brand 5 × Channel 6	—	0	Not sold in this channel
Brand 7 × Channel 6	—	0	Not sold in this channel

All four data-driven skips were previously producing models. Two of them were the source of the disjoint history in the exports.

7.3 What the curve optimizer achieves

Measured across 25 slices, of which 14 have media in the model. Full per-slice results in `outputs/curve_optimizer_experiment.csv`.

Measure	Result
Cross-validated MAPE (what the optimizer optimizes)	Improved **4 of 4** times it fired — mean −2.07 points
Reported holdout (never touched by the search)	Improved **2 of 4** — mean **+0.85 points**, median +0.24
Slices left unchanged	10 of the 14 with media

Slice	Media vars	Curves changed	CV before → after	Holdout before → after
Brand 1 × Ch 1	2	1	12.37 → 11.57	19.64 → 25.51
Brand 1 × Ch 2	1	1	19.60 → 18.22	7.22 → 7.13
Brand 3 × Ch 2	1	1	7.88 → 4.91	6.07 → 6.65
Brand 4 × Ch 2	3	2	7.06 → 3.90	9.84 → 6.90

The honest reading

The optimizer reliably improves the thing it optimizes. Whether that transfers to genuinely unseen data is close to a coin flip on this dataset. It does behave conservatively — it declines to change anything on 10 of the 14 slices with media — and when it fires on a slice with real media weight it can be worth it (Brand 4 × Channel 2 gained 2.94 points of genuine out-of-sample accuracy). But Brand 1 × Channel 1 lost 5.87. **It therefore ships off by default**, as an opt-in tool that reports its cross-validated gain and leaves the analyst to check the holdout.

Why the transfer is weak: media is a small share of these models — the dominant drivers are distribution, seasonality, trade and competitive pressure — and 104 weeks with roughly ten predictors is thin for identifying two extra non-linear parameters per media variable. Robyn and Meridian fit these with Bayesian priors and hierarchical pooling across geographies for exactly this reason: the priors do the work the data cannot. Partial pooling of curve parameters across a brand's channels is the natural next step.

8 Design decisions and rationale

Decision	Alternative rejected	Reasoning
Fixed calendar window anchored to the dataset	Last N rows per slice	The alternative modeled delisted slices on stale history and leaked disjoint due-tos into exports. Data should depend on the window, not the reverse.
Skip thin slices with a stated reason	Model everything, flag afterwards	A model built on 24 selling weeks is not a weak model, it is not a model. But it must be visible, so it is listed with counts and a reason rather than dropped.
Bounded least squares	Ridge regression	Ridge cannot honour sign or box constraints, and a model asserting that price rises lift volume is not deliverable whatever its fit.
Forward stepwise	Lasso, tree importance	Each addition is one testable, explainable decision. Lasso paths are hard to defend line by line; trees do not give signed bounded coefficients.
VIF pruning	Pairwise correlation, PCA	VIF catches redundancy against a *combination* of variables; unlike PCA it preserves raw explainable variables.
Time-based holdout	Random K-fold	Weekly series are autocorrelated; random folds leak the future into training and flatter the model.
Report comparable coefficients	Min-max normalize the inputs	For unregularized constrained least squares, normalization is a linear re-parameterization that changes nothing, while the inverse transform adds real risk. Reporting β×SD solves the actual problem.
Three-way split for curve search	Score on the reported holdout	Scoring on the reported holdout would make every optimized model look better and none of them be better.
Rolling-origin CV with strict dominance	Single inner validation window	A single window over-fitted one quarter: it improved that window while making the untouched holdout worse.
Curve optimization off by default	Optimize automatically in the batch	Measured transfer to unseen data is unreliable. An engine that reports when its own optimization did not generalize is worth more than one that always claims a win.
Cache keyed by every modeling knob	One cache per dataset	Prevents comparing a slice modeled under one setting against slices modeled under another.
Modeling controls in one place	Controls on every screen	Changing a target on a results screen implies a re-model. Results screens filter; they do not model.

9 Verification status

Every claim below is checked by an automated test run against the real dataset.

Check	Result
Sponsor equivalence — our bounded least squares versus Alex's curve_fit implementation	PASS, max relative coefficient difference 7.0×10^{-8}
Decomposition exact — intercept + Σ(β·x) equals the fitted value	PASS, 5.8×10^{-11}
Decomposition still exact with saturation applied	PASS, 1.0×10^{-10}
Due-tos sum to the change in modeled volume	PASS, 6.1×10^{-9}
Scale is a units change only — fitted values unchanged	PASS, 1.2×10^{-10}
Impact / SD invariant to the scale setting	PASS
One calendar window across all 76 models	PASS
Thin slices skipped with reasons; none appear as models	PASS
No pre-window rows in the export	PASS, range is exactly the window
Filter change reads cache without refitting	PASS, ~140 ms
Skipped slice shows a reason, not stale numbers	PASS
Family caps respected across the batch; forced variables survive	PASS
Strictness monotonic in predictor count	PASS, 4.4 / 6.5 / 10.0
Defaults reproduce the previous 76-model baseline exactly	PASS
Hill function: H(k)=0.5, monotone, bounded, H(0)=0, scale-invariant	PASS
Optimizer folds all end before the outer holdout	PASS
Optimizer strict dominance — no fold worse than baseline	PASS
Half-specified or out-of-range curve parameters cleared safely	PASS
Config template round-trip preserves every column	PASS
Dashboard callback wiring — no missing or duplicate component ids	PASS, 78 components, 32 callbacks

10 Known limits and what is not built

10.1 Limits of the current system

- **Curve optimization does not reliably transfer.** Quantified in 7.3. Off by default and documented rather than hidden.

- **No pooling across slices.** Each Brand × Channel is modeled independently. A channel with thin media history cannot borrow strength from its siblings, which is precisely what limits the curve work.
- **Grade thresholds are a team convention** (10% / 40% holdout MAPE) and should be confirmed with the client.
- **Six slices are not modeled** by design. Four of those reflect real data gaps that are worth investigating at source.
- **Model years are 52-week blocks counted back from the anchor**, not fiscal years. An uploaded time map is the way to impose a fiscal calendar.
- **The results cache is in memory.** Restarting the dashboard requires re-running the batch, which takes about 19 seconds.

10.2 Deliberately not built

- **Literal min-max normalization of inputs.** Rationale in 2.10; the comparability problem is solved by reporting β×SD instead.
- **Bayesian priors or hierarchical pooling for media curves.** This is the identified next step for making saturation and decay estimation reliable on data of this length.
- **Automatic curve optimization inside the batch.** Available per-slice on demand; running it unattended across all slices would silently adopt curves that measurably do not generalize.

Companion documents in the repository: `docs/Media_Curves_Math.md` (full derivation and the optimizer experiment), `docs/Action_Items_Alex_Meeting_2026-07-27.md` (sponsor request tracking), `docs/Project_Brief.md`, `docs/Findings.md`.